



PROTOCOL BRIEF

FactorX

A portable commercial identity built from attested source-chain settlements and stored on Creditcoin. Lenders query it. They do not log into FactorX.

Demo	https://factorx.vercel.app
Repository	https://github.com/Ebubechukwucyber/FactorX
Network	Creditcoin Testnet · chain ID 102031 · Attestcoin readability (source chainKey = 1)
Track	DeFi / RWA · BUIDL CTC 2026 Fall

1. Problem

Digital studios, freelancers, and exporters collect on-chain payments from foreign clients. Those settlements are real cashflow. Traditional bureaus and banks do not see them. DeFi credit scores see wallet speculation, not counterparties. Invoice tokenizers often trust an originator's PDF. The missing object is a portable file that proves a commercial payment happened on another chain without a centralized oracle.

2. Product

FactorX is a commercial cashflow passport. A merchant submits a source-chain payment hash and an optional invoice id. Attestcoin attests the source block. FactorX records the receivable on Creditcoin, updates a living score, mints a soulbound passport on first verification, and exposes the file to any contract that can call four view functions.

Request Advance books a credit line equal to 30% of attested volume, reduced by outstanding. It does not transfer tokens. A production lender vault would disburse after reading the same file. MockConsumer is that lender already deployed: it reads score, outstanding, and available room, then emits terms — including a refusal when the line is fully drawn.

3. Attestcoin integration

This project uses Attestcoin **readability** only (writability is out of season scope). Source payments are observed on an EVM origin chain (chainKey 1). The app waits until Attestcoin's attested height covers the payment block, builds header / transaction / Merkle / continuity proofs via the official Proof Builder and USC SDK, runs **verifySingle** against the BlockProver path, then calls FactorX **verifyAndRecord** on Creditcoin.

Replay of the same source transaction is rejected. Self-payments (payer = beneficiary) are rejected. Judges inspect the path in the in-app explorer: source hash, Attestcoin verified, Creditcoin recorded.

4. Architecture

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Contract' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Times-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Contract', textColor=Color(.360784,.419608,.4,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Address (Creditcoin testnet)' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Times-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Address (Creditcoin testnet)', textColor=Color(.360784,.419608,.4,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Role' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Times-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Role', textColor=Color(.360784,.419608,.4,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
ReceivableRegistry	0x1e578b5aE11BEE48361b70470E8FfD939148b7F7	Payer, amount, source tx
AttestcoinVerifier	0x18CD4A1444933E3FCE147fB2e953ECae23e03AD1	Record after attested proof
CommercialScore	0xe90195df4183865CF1533F5B90f15AC37EEbdE02	Living score
FactorCredit	0x96e99678067c62c441152E69975438768e3afEAf	Cap and outstanding
PassportNFT	0xEB1D16bA39D752B5eCABB8D13dA8C8AA364376Ea	Soulbound FXPASS
CommercialIntent	0xB37b771B1337cF555dFAeF8bd7190445D75796aa	Invoice + confidence
MockConsumer	0x2A845d32837CB3549f3e14cE160586961c522AEc	External lender

5. Score and credit

Score starts at 300. It rises with verified payment count, unique counterparties, attested volume, and recency (higher if the last payment is under 30 days). Cap 950. Credit requires score ≥ 400 . Available credit = 30% of volume minus outstanding. A second advance reverts at zero remaining room.

Confidence: payment only = 1; invoice id supplied = 2. That is a commercial-intent flag, not a full invoice-matching engine.

6. How a creditor integrates

No FactorX account. Four reads on Creditcoin: hasPassport(sme), getCommercialScore(sme), outstanding(sme), getAvailableCredit(sme). MockConsumer.checkAndOfferTerms is that integration live. If the line is fully drawn the event message is “No new offer until repay.”

7. What this is not

Not a bank and not a stablecoin vault. Amounts in this demo are source-chain native units (18 decimals), not USD. Repay exists on FactorCredit but is not wired in the UI. Production monetization is lender read-fees first, origination later.

8. Demo path for judges

1. Open <https://factorx.vercel.app> and connect MetaMask to Creditcoin testnet 102031.
2. Submit a source-chain payment hash that Attestcoin has already attested.
3. Open Explorer and follow source tx → Attestcoin → Creditcoin.
4. Confirm the soulbound passport.

5. Check terms and read `BetterTermsOffered` on Blockscout.

FactorX · MIT · Open source · Live testnet contracts listed in the repository README.